

Understanding a **PostgreSQL Cluster** can be a bit confusing because the word “cluster” is used differently in various contexts. Let’s break it down:

What is a PostgreSQL Cluster? (In PostgreSQL Terminology)

In PostgreSQL, a “cluster” is a **collection of databases** managed by a single server instance (i.e., a single `postgres` process). It **does not mean multiple servers or high availability setup** (though it often gets confused with that in general IT).

Think of it like this:

- A **PostgreSQL Cluster** = one **PostgreSQL server instance** + multiple **databases** + shared configurations.

Example:

When you initialize PostgreSQL with:

```
initdb -D /var/lib/postgresql/15/main
```

You’re creating a **PostgreSQL Cluster** at `/var/lib/postgresql/15/main`.

Within this cluster:

- You have the default database: `postgres`
- System database: `template1`
- Possibly `template0`
- You can create your own databases: `mydb1`, `mydb2`, etc.

Each database inside the cluster is **isolated** from the others (you can’t query across them with joins).

Cluster vs Database vs Schema

Term	Description
Cluster	A set of databases managed by a single PostgreSQL instance.
Database	An individual database within the cluster. Each has its own users, tables, etc.
Schema	A namespace inside a database. Schemas help organize objects like tables within one database.

When Does "Cluster" Mean Multiple Servers?

In other contexts, like high availability (HA), replication, or sharding, "cluster" might refer to **multiple PostgreSQL servers working together**. For example:

Type of Clustering	Description
Streaming Replication	One master, one or more read-only replicas.
Logical Replication	Table-level replication.
Patroni Cluster	Highly available PostgreSQL setup using etcd or Consul for leader election.
Citus	PostgreSQL extension for sharding and distributed queries across nodes.

So, in native PostgreSQL terminology, *cluster* = one PostgreSQL server process managing multiple databases.

Summary

-  **PostgreSQL Cluster (officially)** = Single server instance with multiple databases.
-  **Not a cluster** in the sense of multiple servers.

- For HA or distributed setups, "clustering" refers to things like **replication**, **Patroni**, or **Citus**.

Check Status if the postgres cluster is running

Windows

```
C:\Windows\system32>pg_ctl status
pg_ctl: server is running (PID: 15828)
D:/psql_dir/PostgreSQL/16/bin/postgres.exe "-D" "D:/psql_dir/PostgreSQL/16/data"
C:\Windows\system32>
```

Linux

```
postgres@ubuntu-docker-vm:~$ /usr/lib/postgresql/12/bin/pg_ctl status -D /new/location/pgdata
pg_ctl: server is running (PID: 1982)
/usr/lib/postgresql/12/bin/postgres "-D" "/new/location/pgdata"
postgres@ubuntu-docker-vm:~$
```

Start and Stop Cluster Using pg_ctl : Utility provided by Postgresql

Types of pg_ctl Shutdown

- Smart (Default) - Server disallows new connections, but this lets existing connections end their work normally. It shuts down only after transactions: the sessions terminate.
- Fast - The server disallows new connections and aborts their current transactions, active transactions are rolled back and exist gracefully.
- Immediate - Quits/aborts without proper shutdown which lead to recovery on next startup.

Syntax:

```
$pg_ctl _D "<Data Directory>" stop -m <shutdown mode>
```

Start Cluster Syntax:

Windows:

```
C:\Windows\system32>pg_ctl -D "D:/psql/PostgreSQL/16/data" stop
waiting for server to shut down.... done
server stopped

C:\Windows\system32>pg_ctl -D "D:/psql/PostgreSQL/16/data" start
waiting for server to start....2025-07-19 22:32:23.228 IST [8784] LOG:  redirecting log output to logging collector process
2025-07-19 22:32:23.228 IST [8784] HINT:  Future log output will appear in directory "log".
done
server started
```

```
C:\Windows\system32>pg_ctl -D "D:/psql/PostgreSQL/16/data" stop -m fast
waiting for server to shut down.... done
server stopped

C:\Windows\system32>pg_ctl -D "D:/psql/PostgreSQL/16/data" start
waiting for server to start....2025-07-19 22:35:56.611 IST [1352] LOG:  redirecting log output to logging collector process
2025-07-19 22:35:56.611 IST [1352] HINT:  Future log output will appear in directory "log".
done
server started
```

Linux:

```
postgres@ubuntu-docker-vm:/usr/lib/postgresql/12/bin$ /usr/lib/postgresql/12/bin/pg_ctl -D /new/location/pgdata stop -m fast
2025-07-19 22:47:28.836 IST [1916] LOG:  received fast shutdown request
waiting for server to shut down....2025-07-19 22:47:28.891 IST [1916] LOG:  aborting any active transactions
2025-07-19 22:47:28.895 IST [1916] LOG:  background worker "logical replication launcher" (PID 1923) exited with exit code 1
2025-07-19 22:47:28.898 IST [1918] LOG:  shutting down
2025-07-19 22:47:28.950 IST [1916] LOG:  database system is shut down
done
server stopped
postgres@ubuntu-docker-vm:/usr/lib/postgresql/12/bin$ /usr/lib/postgresql/12/bin/pg_ctl -D /new/location/pgdata start
waiting for server to start....2025-07-19 22:47:39.704 IST [1929] LOG:  starting PostgreSQL 12.22 (Ubuntu 12.22-0ubuntu0.20.04.4) on x
86_64-pc-linux-gnu, compiled by gcc (Ubuntu 9.4.0-1ubuntu1-20.04.2) 9.4.0, 64-bit
2025-07-19 22:47:39.705 IST [1929] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2025-07-19 22:47:39.711 IST [1929] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-07-19 22:47:39.738 IST [1930] LOG:  database system was shut down at 2025-07-19 22:47:28 IST
2025-07-19 22:47:39.750 IST [1929] LOG:  database system is ready to accept connections
done
server started
```

Also using SYSTEMCTL Command : Utility provided by Linux

```
postgres@ubuntu-docker-vm:~$ sudo systemctl stop postgresql
postgres@ubuntu-docker-vm:~$ sudo systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
   Active: inactive (dead) since Sat 2025-07-19 23:30:35 IST; 8s ago
   Process: 2127 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2127 (code=exited, status=0/SUCCESS)

Jul 19 23:30:03 ubuntu-docker-vm systemd[1]: Starting PostgreSQL RDBMS...
Jul 19 23:30:03 ubuntu-docker-vm systemd[1]: Finished PostgreSQL RDBMS.
Jul 19 23:30:35 ubuntu-docker-vm systemd[1]: postgresql.service: Succeeded.
Jul 19 23:30:35 ubuntu-docker-vm systemd[1]: Stopped PostgreSQL RDBMS.
postgres@ubuntu-docker-vm:~$ sudo systemctl start postgresql
postgres@ubuntu-docker-vm:~$ sudo systemctl status postgresql
• postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled; vendor preset: enabled)
   Active: active (exited) since Sat 2025-07-19 23:30:56 IST; 3s ago
   Process: 2142 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 2142 (code=exited, status=0/SUCCESS)

Jul 19 23:30:56 ubuntu-docker-vm systemd[1]: Starting PostgreSQL RDBMS...
Jul 19 23:30:56 ubuntu-docker-vm systemd[1]: Finished PostgreSQL RDBMS.
postgres@ubuntu-docker-vm:~$
```

Reload

```
postgres@ubuntu-docker-vm:/usr/lib/postgresql/12/bin$ /usr/lib/postgresql/12/bin/pg_ctl -D /new/location/pgdata reload
server signaled
2025-07-19 22:54:09.196 IST [1929] LOG:  received SIGHUP, reloading configuration files
```

```
postgres=# SELECT pg_reload_conf();
2025-07-19 22:58:55.428 IST [1929] LOG:  received SIGHUP, reloading configuration files
 pg_reload_conf
-----
 t
(1 row)
```

means that the server configuration was successfully reloaded. Here's a breakdown:

- `pg_reload_conf()` is a PostgreSQL function that instructs the server to reload its configuration files (`postgresql.conf`, `pg_hba.conf`, etc.) without restarting the database.
- The result `t` (short for "true") confirms that the reload command was accepted and executed successfully.
- If it had failed (e.g., due to permission issues or syntax errors in the config files), it would return `f` (false), or an error would be thrown.

RESTART

Windows

```
C:\Windows\system32>pg_ctl restart
waiting for server to shut down.... done
server stopped
waiting for server to start....2025-07-19 23:01:47.933 IST [15828] LOG:  redirecting log output to logging collector process
2025-07-19 23:01:47.933 IST [15828] HINT:  Future log output will appear in directory "log".
done
server started
C:\Windows\system32>
```

Linux

```
postgres@ubuntu-docker-vm:~$ /usr/lib/postgresql/12/bin/pg_ctl restart
pg_ctl: no database directory specified and environment variable PGDATA unset
Try "pg_ctl --help" for more information.
postgres@ubuntu-docker-vm:~$ /usr/lib/postgresql/12/bin/pg_ctl restart -D /new/location/pgdata
waiting for server to shut down....2025-07-19 23:05:00.596 IST [1929] LOG:  received fast shutdown request
2025-07-19 23:05:00.641 IST [1929] LOG:  aborting any active transactions
2025-07-19 23:05:00.645 IST [1929] LOG:  background worker "logical replication launcher" (PID 1936) exited with exit code 1
2025-07-19 23:05:00.648 IST [1931] LOG:  shutting down
2025-07-19 23:05:00.723 IST [1929] LOG:  database system is shut down
done
server stopped
waiting for server to start....2025-07-19 23:05:00.824 IST [1982] LOG:  starting PostgreSQL 12.22 (Ubuntu 12.22-0ubuntu0.20.04.4) on x
86_64-pc-linux-gnu, compiled by gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, 64-bit
2025-07-19 23:05:00.826 IST [1982] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2025-07-19 23:05:00.834 IST [1982] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2025-07-19 23:05:00.867 IST [1983] LOG:  database system was shut down at 2025-07-19 23:05:00 IST
2025-07-19 23:05:00.886 IST [1982] LOG:  database system is ready to accept connections
done
server started
postgres@ubuntu-docker-vm:~$
```

How to check if changing a parameter requires a restart or not

Get into the Data Directory and check the Postgresql.conf file, It is clearly mentioned in the file which parameter change will require a restart as shown in the snippet appended below:

```
postgres=# show data_directory;
      data_directory
-----
 /new/location/pgdata
(1 row)

postgres=# \q
postgres@ubuntu-docker-vm:~$ cd /new/location/pgdata
postgres@ubuntu-docker-vm:/new/location/pgdata$ vi postgresql.conf
```

```
#port = 5432                                # (change requires restart)
max_connections = 100                       # (change requires restart)
#superuser_reserved_connections = 3         # (change requires restart)
#unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
                                              # (change requires restart)
#unix_socket_group = ''                    # (change requires restart)
#unix_socket_permissions = 0777           # begin with 0 to use octal notation
                                              # (change requires restart)
#bonjour = off                            # advertise server via Bonjour
                                              # (change requires restart)
#bonjour_name = ''                        # defaults to the computer name
                                              # (change requires restart)
```

```
#work_mem = 4MB                            # min 64kB
#maintenance_work_mem = 64MB              # min 1MB
#autovacuum_work_mem = -1                 # min 1MB, or -1 to use maintenance_work_mem
#temp_buffers = 8MB                      # min 128kB
```